# TEKNOFEST

## HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

## UÇAN ARABA YARIŞMASI

## FİNAL TASARIM RAPORU



**TAKIM ADI: Kelebek**

**TAKIM ÜYELERİ:**

**Mustafa Emre Zeyrek**

**Ahmet Gül**

**Haktan Ergül**

**Ersin Ortagenç**

**Mustafa Sercan Pay**

# İçindekiler

1. GİRİŞ ..... 3
2. ÇÖZÜM ..... 4
2.1. Programlama Dili ve Sensörler ..... 4
2.2. Senaryo 1'de, B Noktasına Gidiş Algoritması ..... 6
2.3. Senaryo 4'te, Yasak Bölgeden Kaçış Algoritması ..... 7
2.4. Senaryo 7'de, Trafikte Güvenli Uçuş Algoritması ..... 8
2.5. Senaryo 8'de, En Yakın Hastaneyi Bulmayı Sağlayan Algoritma ..... 9
2.6. Senaryo 10 İçerisinde GNSS Arızası Yaşandığında İnişi Tamamlayan Algoritma ..... 9
2.7. Senaryo 11 İçerisinde GNSS Arızası Yaşandığında Görevi Tamamlayan Algoritma ..... 10
2.8. Senaryo 12'de GNSS Arıza Durumunu Tespit Eden Algoritma ..... 11
2.9. Senaryo 13'te İniş İçin Sensör Seçimini Yapan Algoritma ..... 12
2.10. Senaryo 15'te Manyetometre Çalışmadığında Aracı Yönlendiren Algoritma ..... 13
2.11. Senaryo 16'da Şarj Durumu ve Şarj İstasyonlarının Tespiti ..... 14
3. SONUÇ ..... 15
4. KAYNAKÇA ..... 16

# 1. GİRİŞ

Otonom araçlar, yapay zeka ve türevi olan algoritmalar sayesinde kendi kendine hareket edebilir ve kritik görevleri yerine getirebilir aşamaya ulaşmıştır. Her ne kadar daha alınacak epey yol olsa da otonomi ve akıllı makineler çok yakında yardımlaşarak, öğrenerek ve hatta öğreterek, belirsizlik durumlarında insandan daha güçlü kararlar verebilecek hale geleceğini görmek hiç zor değildir.

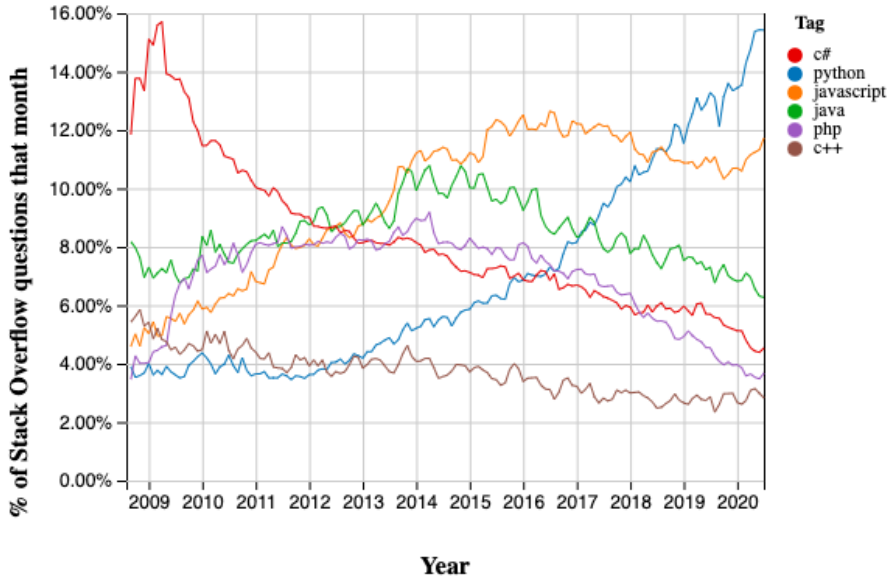
Bu raporda bir çok ulaşım faaliyine yeni bir çehre kazandırmak için geliştirilmekte olan Uçan Araba Cezeri'nin otonomlaşması için Teknofest tarafından belirlenen senaryolar özelinde çalışmalar yapılmıştır. Senaryolar sensör bozulmaları, acil durum inişleri ve zorlu hava koşulları gibi bir uçan arabanın gerçek hayatta karşılaşımlası durumunda yüksek risk içeren durumlar çevresinde toplanmıştır.

Teknofest tarafından yayımlanan rapor şablonu çok nettir. Raporun genel gidişatı şablonu takip etmiştir ve ilgili sorulara cevap vermek üzerine hazırlanmıştır.

# 2. ÇÖZÜM

## 2.1. Programlama Dili ve Sensörler

- Kodları yazarken Pyhton proglamlama dilini kullandık. Bunun sebebi, yapay zeka ile ilgili işlevlerin uygulanmasını önemli ölçüde kolaylaştıran mükemmel makine öğrenimi kütüphanesine sahip olmasıdır. Aynı zamanda grubumuz makine öğrenimi konusunda derin tecrübeye sahip değildir. Python , öğrenme sürecinde ilk başarılı sonuçlarımızı almamız için gereken süreyi önemli ölçüde azaltan, tam özellikli bir kütüphane hattı sunan minimalist ve sezgisel bir dildir. Aşağıdaki grafikten de yapabileceğimiz çıkarıma göre yazılımcılar son yıllarda bu dile merak salmıştır.




(Yıllara göre yazılım dilleri hakkında sorulan soru sayısı grafiği **[1]**)

- Pyhton yerine tercih edebileceğimiz bir diğer programlama dili ise C++'dır. C++ programlama dili makine öğrenmesinde gelişmiş bir altyapıya sahip olup özellikle değişkenleri matris formunda belirlenmesini desteklemektedir. Ayrıca gelişmiş kütüphane yapısı bir çok farklı alanda çalışma yapabilme imkanını sunmaktadır. C++ uzun süredir makine öğrenmesi alanında kullanılması farklı hataların ayıklanmasını ve hata oranın az olmasını sağlamıştır. Otonom sürüş konusunda hata payının az olması en önemli isterlerin başında gelmektedir ki bu özellik de dili avantajlı kılmaktadır.**[2]**

- Sensörün etrafına sarılan tel sayesinde basıncın neden olduğu dirençteki değişimi voltaj cinsinden gözlemleriz ve bu değeri basınca değiştirerek deniz mesafesinden yüksekliği elde ederiz. GNSS ve barometre irtifa verilerinin farkını anlamamız için öncelikle GNSS sisteminden bahsetmeliyiz. GNSS, bulunduğumuz yüksekliği uydular ve sensörler aracılığı ile ölçen bir sistemdir. Bu yükseklik deniz mesafesinden itibaren olarak kabul edilir fakat aslında durum öyle değildir. Dünya'da denizlerin yükseklikleri aynı değildir bu yüzden bilim insanları GNSS sistemini tasarlarken dünyayı düz bir şekil olarak kabul etmişlerdir ve GNSS uydusu için bir referans yeri belirlemişlerdir. Bizim sistemimiz ile referans veri arasındaki fark da bize yüksekliğimizi verir. Baktığımız zaman bu mutlak yükseklik değildir ama eğer tüm sistemler için aynı noktayı referans alırsak mutlak yükseklikleri doğru olmasa bile kendileri aralarındaki bağıl yükseklik aynı olur bu da sistemin doğru çalışmasını sağlar. Barometre sensörü ise mutlak yüksekliği verir. Yukarda da bahsettiğimiz gibi, yüksekliği basınç farkından yararlanarak ölçer ve bir referans noktası yoktur.

- Dönüölçer ve ivmeölçer hataları kabaca ikiye ayrılarak incelenebilir. Bunlar, "gürültü" olarak adlandırılan doğru değer etrafındaki hızlı değişimler ve "sabit kayma kararsızlığı" olarak adlandırılan, doğru değerden farktır. Sabit kayma kararsızlığı, dönüölçerler için derece/saat, ivmeölçerler için yerçekimi ivmesi (g) cinsinden ifade edilir.İvmeölçer bir kütlenin sahip olduğu ivmeyi ölçmek için kullanılan bir cihazdır. İtici sistemden ve fiziksel limitlerden dolayı, ivme ölçen bu sensörlerdeki en önemli durum yer çekiminden etkilenmeleridir.**[3]**

  Dönüölçer ve ivmeölçer sistemlerinin diğer bir dezavantajı ise, konum, hız ve yönelim çözüm hatalarının zamanla orantılı bir biçimde artmasıdır. Zamanla artan bu hataların nedeni; ivmeölçer ve dönüölçerlerin yaptıkları ölçüm hataları ile hız, konum ve yönelim başlangıç değerlerindeki hatalardır. Bu gibi durumlarda ivmeölçer ve dönüölçer sistemlerinin VOR, TACAN, LORAN, GPS, Odometre, Barometre, Hava hızı sensörü gibi "dış destek kaynakları" tarafından desteklenmesi gerekebilir.**[4]**

## 2.2. Senaryo 1'de, B Noktasına Gidiş Algoritması

- Hava aracının hedefe hareketini sağladığımız algoritma:

| **Algoritma** | **Açıklama** |
|---|---|
| a=cezeri.gnss.enlem<br>b=cezeri.gnss.boylam<br>c= varis_bolgesi.enlem<br>d= varis_bolgesi.boylam | Bu şekilde bulunduğumuz konumun ve hedef konumun koordinatlarını elde ettik. |
| if d<b:<br>cezeri.don(-3.14/2)<br>cezeri.bekle(1.05)<br>cezeri.dur()<br>aci=math.atan((abs(c-a))/(abs(d-b)))<br>cezeri.don(-aci)<br>cezeri.bekle(1)<br>cezeri.dur() | Yönü belirlememiz için iki koordinat arasındaki açıyı bilmemiz gerekmekte. Eğer hedef bölgenin boylamı şuan bulunduğumuz konumun boylamından küçükse yani bölge aşağımızdaysa öncelikle uçağı 90 derece saat yönünde döndürerek y eksenine paralel hale getiriyoruz. Sonrasında ise iki nokta arasındaki açıyı bularak bir o kadar daha saat yönünde döndürüyoruz ve yönümüz belirlenmiş oluyor. |
| İf d>b:<br>cezeri.don(3.14/2)<br>cezeri.bekle(1.05)<br>cezeri.dur()<br>aci2=math.atan((abs(c-a))/(abs(d-b)))<br>cezeri.don(aci2)<br>cezeri.bekle(1)<br>cezeri.dur() | Aynı şekilde hedef boylam güncel boylamdan büyükse bu sefer de saat yönünün tersine 90 derece çevirerek aracı y eksenine paralel yapıyoruz. Ve yine iki nokta arasındaki açıyı belirliyerek bir o kadar daha saat yönünün tersinde döndürüyoruz. ( iki kodu aynı anda kullanırken burada if değil else kullanacağız). |

- "cezeri.hedefler" kodu ve "bolge.enlem" "bolge.boylam" kodları ile hava aracının gitmesi gereken koordinatları öğrenebiliriz. **[5]**
- Hava aracı harita üzerinde sadece kendisi için ayrılan özel iniş bölgelerine inebilir. Bu bölgeler dışında bölgelere arıza durumu veya olası diğer acil durumlar dışında iniş yapılmamalıdır. Bu iniş bölgeleri simülasyonda şu şekillerde belirlenmiştir: hastane, şarz istasyonu, iniş bölgesi.

- İnişte irtifa tespiti için birkaç farklı kod kullanılabilir, güvenlik için bunların tamamından alınan değerlerin karşılaştırılması gerekir. Bu kodlar sırasıyla "cezeri.barometre.yukseklik", "cezeri.barometre.yukseklik" ve "cezeri.lidar.mesafe"'dir.
- Aracın 3 farklı hız seçeneği var. Bunlar hızlı, yavaş ve orta. İnişlerde aracın hızı yavaş olmalı ve bu da simülasyonda cezeri.hiz(yavas) ile 6m/s olarak ayarlanmış durumdadır.

## 2.3. Senaryo 4'te, Yasak Bölgeden Kaçış Algoritması

- "bolge.ucusa_yasakli_bolge" değişkeni haritada tespit edilmek istenen yasaklı bölgelerin bilgisini vermektedir. Örnek olarak güneye doğru hareket eden uçan aracın bir bölge önündeki yerin yasaklı bölge olup olmadığını öğrenmek için:

  "bolge = cezeri.harita.bolge(cezeri.gnss.enlem, cezeri.gnss.boylam - 1)" olarak belirledğimiz "bolge" değişkenini "bolge.ucusa_yasakli_bolge = True yada False" olarak tanımlayıp, elde edebiliriz.

- Bu bölgelere girmemek için tasarladığımız algoritma :

| Algoritma | Açıklama |
|---|---|
| a = cezeri.gnss.enlem<br>b = cezeri.gnss.boylam<br>bolge = cezeri.harita.bolge(a,b-1)<br>bolge2 = cezeri.harita.bolge(a-1,b) | Başlangıçta güneye doğru hareket eden hava aracının havaalanını tarafındaki bir önünde ve bir sağında yer alan bölgelerin verisi çekilir. |
| If bolge.ucusa_yasakli_bolge = True:<br>cezeri.dur()<br>cezeri.sola_git(cezeri.hizli) | Eğer hava aracının bir bölge önünde (yani hava alanının olduğu tarafta) uçuşa yasaklı bölge varsa hava aracı sola doğru hareket eder. |
| If bolge2.ucusa_yasakli_bolge = True:<br>cezeri.dur()<br>cezeri.ileri_git(cezeri.hizli) | Eğer hava aracının bir bölge sağında (yani hava alanının olduğu tarafta) uçuşa yasaklı bölge varsa hava aracı ileri doğru hareket eder. |

## 2.4. Senaryo 7'de, Trafikte Güvenli Uçuş Algoritması

- Öncelikli olarak cezeri.trafik komutu ile trafiğin varlığı tespit edilebilir. Kesintisiz bir ulaşım isteniyorsa bölge.trafil_bolgesi komutu kullanılarak cezeri.trafik komutundan gelen veri saklanıp yoğun trafikte hedefe kesintisiz varışın tamamlanabileceği irtifa seviyesi belirlenir. Sonrasında belirlenen irtifa seviyesinde güvenli ulaşımın sağlanması için cezeri.irtifa_araliği komutu kullanılır. Doğru irtifada olduğunun kontrolü için cezeri.gnss.irtifa komutu kullanılabilir.
- Hava araçları ile kaza yapmamak için tasarladığımız algoritma :

| **Algoritma** | **Açıklama** |
|---|---|
| bolge1=cezeri.yerel_harita[0]<br>bolge2=cezeri.yerel_harita[1]<br>bolge3=cezeri.yerel_harita[2] | Diagnostik panelde baktığımızda sol altta bir panel bulunmakta. Buna yerel harita deniyor ve biz tam ortasında bulunuyoruz. Önümüzdeki üç bölgenin bölgelerini yerel harita verisinden soldaki kodlar sayesinde çektik. |
| while cezeri.aktif():<br>varis_bolgesi = cezeri.hedefler[0]<br>bolge1=cezeri.yerel_harita[0]<br>bolge2=cezeri.yerel_harita[1]<br>bolge3=cezeri.yerel_harita[2]<br>if bolge1.trafik== True or bolge2.trafik==True or bolge3.trafik== True:<br>cezeri.dur()<br>else:<br>cezeri.ileri_git(cezeri.hizli) | Sonrasında ise bu üç bölgeden herhangi birinde trafik olması durumunda cezeri.dur() komutunu kullanarak arabayı durdurduk. Trafik olmadığı zaman ise aracımız ilerlemeye cezeri.ileri_git(cezeri.hizli) komutu ile devam etti. Trafik sürekli değiştiği için de kodumuzu while döngüsüne yazarak trafiği sürekli kontrol ettik. |

## 2.5. Senaryo 8'de, En Yakın Hastaneyi Bulmayı Sağlayan Algoritma

- "cezeri.harita.hastaneler" dizisi, haritada bulunan hastanelere ait bilgileri içermektedir. Örnek olarak cezeri.harita.hastaneler[0].enlem ile ilk hastanenin enlem bilgilerine derece cinsinden ulaşabiliriz.
- En yakın hastanenin seçimini yaptığımız algoritma:

| Algoritma | Açıklama |
|---|---|
| for hastane in cezeri.harita.hastaneler:<br>print(hastane.enlem, hastane.boylam) | Bu kod yapısı sayesinde cezeri.harita.hastaneler veri yapısındaki hastanelerin konumlarını elde ettik |
| a= cezeri.gnss.enlem<br>b= cezeri.gnss.boylam<br>c=cezeri.harita.hastaneler[0].enlem<br>d=cezeri.harita.hastaneler[0].boylam<br>e=cezeri.harita.hastaneler[1].enlem<br>f=cezeri.harita.hastaneler[1].boylam<br>g=cezeri.harita.hastaneler[2].enlem<br>h=cezeri.harita.hastaneler[2].boylam | (a,b) bizim koordinatımızı,<br>(c,d) ilk hastanenin koordinatını,<br>(e,f) ikinci hastanenin koordinatını,<br>(g,h) üçüncü hastanenin koordinatını bizlere verir. |
| u1= math.sqrt((c-a)*(c-a)+(d-b)*(d-b))<br>u2= math.sqrt((e-a)*(e-a)+(f-b)*(f-b))<br>u3= math.sqrt((g-a)*(g-a)+(h-b)*(h-b)) | u1,u2 ve u3 verilerine koordinatların kareleri toplamının karekökünü alma kodu ile ulaştık ve mesafelerimizi belirledik. |
| liste= [u1,u2,u3]<br>en_kisa= min(liste) | Uzaklık verilerini ''list'' komutu ile listeledik ve ''min'' komutu ile en ufak olan mesafeyi belirledik. |

## 2.6. Senaryo 10 İçerisinde GNSS Arızası Yaşandığında İnişi Tamamlayan Algoritma

- GNSS sensörünün çalışma durumu hakkında bilgi veren cezeri.gnss.hata kodunu kullandık. Değerin ''True'' olması ile hata olduğunu anladık.
- 
- Bu arıza sonrası hava aracının irtifasını hesapladığımız algoritma:

| Algoritma | Açıklama |
|---|---|
| cezeri.gnss.hata | Bize gnss'de hata olduğu gösteren kod bu. Eğer bu kod True değerini döndürür ise gnss arızalı demektir. |
| if cezeri.gnss.hata== True:<br>yukseklik2=cezeri.barometre.yukseklik | Bu algoritma sayesinde GNSS'de hata olması durumunda yüksekliğin barometredeki yüksekliğe eşit olduğunu belirtiyoruz. |

## 2.7. Senaryo 11 İçerisinde GNSS Arızası Yaşandığında Görevi Tamamlayan Algoritma

- Bu arıza sonrası hava aracının pozisyonunu(yatay) hesapladığımız algoritma:

| Algoritma | Açıklama |
|---|---|
| If cezeri.gnss.hata = True:<br>son_enlem= cezeri.gnss.enlem<br>son_boylam= cezeri.gnss.enlem<br>X1= cezeri.harita.bolge(son_enlem-1)<br>X2 = varis_bolgesi.enlem+1<br>Y1= cezeri.harita.bolge(son_boylam-1)<br>Y2 = varis_bolgesi.boylam+1<br>X = X2-X1<br>Y = Y2-Y1<br>Yol =math.sqrt((X2)$^2$+(Y2)$^2$) | Gnss sensöründe hatayı tespit ettiğimiz anda hata tespit edilen bölgeden bir önceki yani en son doğru tespit edilen enlem ve boylam bilgileri alınır. Sonrasında varış bölgesinin enlem ve boylam bilgileri çekilir fakat hava aracı son doğru tespit edilen enlem ve boylam bölgesinden 1 bölge önde olduğu varış bölgelerinin enlem ve boylam değerlerine +1 eklenir. Ardından birbirlerinin farkı alınarak kalan yol enlem ve boylam olarak bulunur. Sonunda hipotenüs bulunarak gitmesi gereken en kısa yol bulunur. |
| cezeri.ileri_git(cezeri.hizli)<br>cezeri.bekle(yol/12)<br>cezeri.dur()<br><br>cezeri.saga_git(cezeri.hizli)<br>cezeri.bekle(yol/12)<br>cezeri.dur() | Hava aracının doğru yere iniş yapabilmesi için hesaplanan kalan yol aracın hızı olan 12 m/s'ye bölünerek hava aracının uçması gereken süre bulunur ve bu süre sonunda hedef varış noktasına ulaşması beklenir. Hedef varış noktası, kalkış noktasının sağ çaprazında olduğu varsıyılmıştır. |

## 2.8. Senaryo 12'de GNSS Arıza Durumunu Tespit Eden Algoritma

- Gnss karıştırma (jamming) temelde sinyali kesmeye yönelik saldırı iken, Gnss yanıltma (spoofing) güncel sinyali saptırmaya yöneliktir. Örneğin, uçan arabanın konumunu göremiyosanız bu bir Gnss karıştırma (jamming) saldırısı olabilir ama uçan arabanın konumunu yanlış görüyosanız bu Gnss yanıltma (spoofing) saldırısıdır. Gnss yanıltma saldırısı sonucunda konumu yanlış okuyan otonom sistem yanlış yöne hareket eder.[**6][7]**
- Gnss'in spoofing arızası verip vermediğini anlamak için aracın konumunu farklı bir yöntemle bulmamız gerekiyor. Bu bulduğumuz ikinci konum verisini gnss'den aldığımız veri ile karşılaştırarak hatayı tespit edebiliriz. Gereken ikinci konum verisini bulabilmek için yol=(hız)x(zaman) temel formülünden yararlandık. Aracın hızı ve hareket süresini bu işlemden geçirerek aldığı mesafeyi, ve bu şekilde de bulunması gereken konumu analitik olarak hesapladık. GNSS yanıltma (spoofing) durumunu tespit ettiğimiz algoritma:

| • **Algoritma** | **Açıklama** |
|---|---|
| hedef_konum = cezeri.hedefler[0] | Hedef konum için değişken atanır |
| gnss_spoofing = false | GNSS spoofing arızası başta sıfırlanır. |
| hedef= cezeri.hedefler[0]<br>a_enlem=cezeri.gnss.enlem<br>time_enlem=(hedef_konum.enlem-a_enlem)/12 | Aracın boylam boyunca hareketinin süresi hesaplanır. Enlem için de aynı işlem tekrarlanır. |
| if time_enlem>0<br>cezeri.ileri_git(cezeri.hizli)<br>cezeri.bekle(1)<br>time_enlem=time_enlem-1 | Aracın hareketi her saniye takip edilir. Yol hız zaman eşitliği ile aracın bulunması gereken konum matematiksel olarak hesaplanır ve çıkan sonuç ile GNSS'den okunan sonuç karşılaştırılır. |

| | |
|---|---|
| a_enlem=a_enlem+12<br>if a_enlem=cezeri.gnss.enlem<br>gnss_spoofing = false<br>else<br>gnss_spoofing = true | Sonuçlar farklıysa gnss_spoofing arıza bayrağı kaldırılır. |

## 2.9. Senaryo 13'te İniş İçin Sensör Seçimini Yapan Algoritma

- İniş için öncelikle Lidar sensöründen faydalandık. İniş öncesi lidar sensörü iniş anında oluşabilecek engelleri tespit etmek için uygun bir sensördür. Lazer altyapı sisteminin olması diğer sistemlere göre daha güvenilir sonuçlar vermektedir.
- Sistemdeki diğer sensörlerin dikey hatta hata yüzdesi daha fazla olduğu için iniş kısmında kullanılmaları uygun değildir. Bunun temel sebebi diğer sensörlerin ölçümü hız ve ivme olarak konuma dayalı olmasıdır. Yakındaki bir engeli tespit edebilme bu sensörlerin yapısına aykırıdır.

- Bu seçimi yapan algoritma:

| • **Algoritma** | **Açıklama** |
|---|---|
| cezeri.gnss.irtifa,cezeri.barometre.yukseklik, cezeri.radar.mesafe , cezeri.lidar.mesafe | Bize yüksekliğimiz hakkında bilgi veren kodlar bunlar. |
| cezeri.gnss.hata,cezeri.barometre.hata, cezeri.radar.hata, cezeri.lidar.hata | Bunlar ise bu sensörlerin hatasını tespit eden kodlar |
| if cezeri.gnss.hata== True and cezeri.radar.hata== True and cezeri.barometre.hata== True:<br>yeni_yukseklik=cezeri.lidar.mesafe | Tüm sensörlerin aynı anda hata vermesini ''and'' koşulu ile birbirine bağladık. Eğer sensörler True değerini verirse hata var demektir ve if koşulu sayesinde yeni yüksekliğimiz cezeri.lidar.mesafe'ye eşit olur ve işlemlerimizi buna göre yaparız |

| | |
|---|---|
| | |

## 2.10. Senaryo 15'te Manyetometre Çalışmadığında Aracı Yönlendiren Algoritma

- Manyetometrenin veri kaybını tespit ettiğimiz komut cezeri.manyetometre.hata komutudur. Eğer değer ''True'' ise sensörde arıza var demektir. Aynı zamanda IMU sensörüne gürültü binmiş ve verilerin güvenilirliği azalmış olabilir. Sensördeki gürültüyü tespit ettiğimiz komut cezeri.manyetometre.gurultu komutudur. Değer ''True'' ise aldığımız değerlerin güvenilirliliği düşüktür.
- Aracın gitmesi gereken yön ve gittiği yön, manyetometre bozuk olduğu için birbirinden farklı olacaktır. Gitmesi gereken yönü koordinatlar sayesinde elde edebilirken gittiği yönü de yatay ve enlemdeki hızına bakarak elde edebiliriz.

| **Algoritma** | **Açıklama** |
|---|---|
| c=varis_bolgesi.enlem<br>d=varis_bolgesi.boylam | Varış bölgesinin koordinatlarını bu kodlarla elde edebiliriz. Bir kere elde etmemiz yeterli çünkü varış bölgesi değişmiyor. |
| while cezeri.aktif():<br>varis_bolgesi = cezeri.hedefler[0]<br>if cezeri.manyetometre.hata==True:<br>a=cezeri.gnss.enlem<br>b=cezeri.gnss.boylam | İkinci ihtiyacımız olan şey manyetometre verisi bozulduğunda bulunduğumuz konum olduğu için hata kodunu if ile True'ya eşitleyip bu koşul sağlandığında enlem ve boylam bilgisini çekiyoruz. Dediğimiz gibi manyetometre verisi bozulduğunda bu koordinatlara ihtiyacımız var bu yüzden while döngüsünün içinde sürekli kontrol ediyoruz. |
| yon1= math.atan((abs(d-b))/(abs(c-a))) | Gitmemiz gereken yönü yukarıda da bahsettiğimiz gibi ilk ve son koordinatlardan rahatlıkla bulabiliriz. Onu da math.atan fonksiyonu ile belirliyoruz. |

- Yukarıda da bahsettiğimiz üzere gittiğimiz yön ise hızlarımızın oranının arctan değerine eşit olur.

| Algoritma | Açıklama |
|---|---|
| Vx=cezeri.imu.Vx<br>Vy=cezeri.imu.Vy | Aracımızın enlem ve boylamdaki hızlarını bu şekilde elde ediyoruz. |
| while cezeri.aktif():<br>varis_bolgesi = cezeri.hedefler[0]<br>if cezeri.manyetometre.hata==True:<br>Vx=cezeri.imu.Vx<br>Vy=cezeri.imu.Vy<br>yon2= math.atan(abs(Vy)/abs(Vx)) | Manyetometre verimiz bozulduğunda elde ettiğimiz hız bilgilerinden boylam hızını enlem hızına oranlayıp çıkan değeri arctan fonksiyonuna yerleştirirsek yönelim bilgimizi radyan cinsinden elde etmiş oluruz. |

## 2.11. Senaryo 16'da Şarj Durumu ve Şarj İstasyonlarının Tespiti

- Batarya ile ilgili bilginin doğruluğunu teyit etmek için önce cezeri.batarya.hata komutunu kullanıp ''False'' bilgisini aldık ve sensörün çalışır durumda olduğunu anladık. Devamında cezeri.batarya.veri komutuyla uçan arabanın batarya doluluk bilgisini elde ettik.
- Batarya doluluk oranı %20'nin altına düştüğü zaman senaryolar başarısız sayılmaktadır. Bu yüzden en yakın şarj istasyonuna giderken harcayacağımız yüzde şarj miktarına, %20 şarj yüzdesini ekledik. Uçan arabanın havadayken şarjı biterse büyük bir kayıp yaşanacağı için toplam değeri de bir güvenlik faktörüyle çarpma kararı alarak finalde %50 doluluk oranına düşünce şarj istasyonuna yönelme kararı aldık.
- Şarj istasyonları hakkında konum bilgilerini elde etmek için cezeri.harita.sarj_istasyonlari[ ] dizisini kullandık.

  **for** sarj **in** cezeri.harita.sarj_istasyonlari :

  **print**(sarj.enlem,sarj.boylam)

  **print**(''----------''')

  Yukarıdaki kodu kullanarak şarj istasyonlarının enlem ve boylam bilgilerine ulaştık.**[8]**

# 3. SONUÇ

Bu rapor sonucunda toplamda 10 adet senaryo için çalışmalar sunulmuştur. Verilen cevaplar genellikle senaryoların en önemli kısımlarıyla ilgilidir. Sensörler ve oluşabilecek hata tipleri araştırılmıştır. Aynı zamanda güncel olarak makine öğrenmesi alanında kullanılan programlamla dilleri hakkında bilgiler verilmiştir.

Teknofest'in Uçan Araba Yarışması yapısını mekanik tasarım konusundan çıkarıp otonomiye yöneltmesi de Kelebek Takımı'nın vizyonunu genişletmiştir. Uçan arabaların fonksiyonları göz önünde bulundurulduğunda oluşabilecek güvenlik zaafiyetleri günümüzdeki kara araçlarından çok daha yüksek seviyededir. Bu bağlamda uçan arabalardaki otonom sistemler mükemmel olmalı ve riskleri minimuma indirmelidir. Bir uçan arabanın sahip olduğu otonom sistem teknik özelliklerinden daha önemlidir.

# 4. KAYNAKÇA


- **[1]** https://stackoverflow.com/
- **[2]** https://www.section.io/engineering-education/an-introduction-to-machine-learning-using-c++/
- **[3]** https://www.elektrikport.com/universite/inertial-measurement-unit-(imu)-nedir/17369#ad-image-0
- **[4]** https://www.mmo.org.tr/sites/default/files/a5b1e4daae265b7_ek.pdf
- **[5]** Riders Teknofest'22 Uçan Araba Yarışması Eğitimi, Araç Komutları
- **[6]** https://www.maritimeglobalsecurity.org/media/1043/2019-jamming-spoofing-of-gnss.pdf
- **[7]** https://www.sciencedirect.com/science/article/pii/S2405959520303040#:~:text=Jamming%20attacks%20can%20degrade%20the,detection%20of%20GNSS%20jamming%20signals
- **[8]** TEKNOFEST 2022 Uçan Araba Yarışması 2. Gün Eğitimi